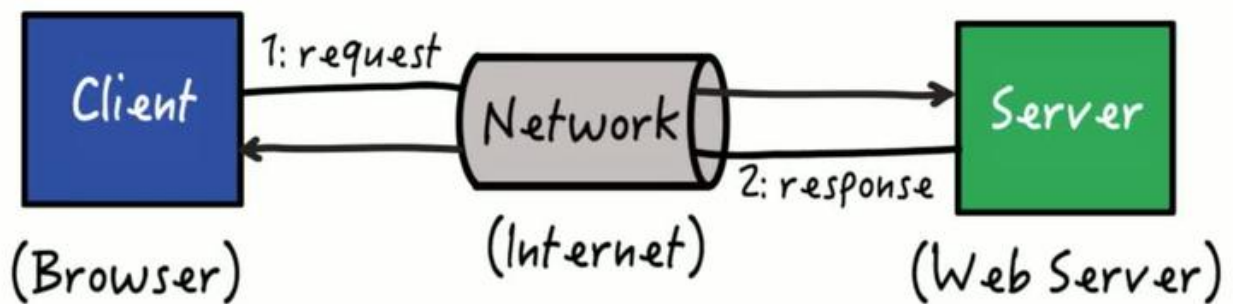


Hypertext Transfer Protocol (HTTP)



ANALISIS CABECERAS HTTP

Despliegue de aplicaciones

José Miguel Verdú Sagredo

Contenido

1. Tarea 1. Investiga y responde	2
1.1. ¿En qué arquitectura está basada el protocolo HTTP?	2
1.2. ¿Qué tipos o categorías de códigos de estado existen? Enuméralos y explica brevemente en qué consisten.	2
1.3. Indica algún ejemplo de web con error 4XX.	3
1.4. Si obtenemos como respuesta HTTP "HTTP/1.1 302 Found", ¿qué tipo de error es y qué significa?	3
1.5. Si obtenemos como respuesta HTTP "HTTP/1.1 400 Bad Request", ¿qué tipo de error es y qué significa?	3
1.6. Si obtenemos como respuesta HTTP "HTTP/1.1 401 Authorization Required", ¿qué tipo de error es y qué significa?	3
2. Tarea 2. Inspección de las cabeceras HTTP	4
2.1. Qué cabeceras de respuesta del servidor se generan en cada caso y para qué sirven. Además, indica qué protocolo de transmisión utilizan una vez finalizada la carga del sitio web.	4
2.2. Las fuentes o "source" de las webs. Explica con tus palabras qué ves en la estructura de carpetas. Acompaña de alguna captura tu explicación.	6
2.3. Qué diferencia hay entre la información que viaja transmitida con el protocolo HTTP y el HTTPS.	7
2.4. Indica en qué sección de la herramienta de inspección que utilices se pueden revisar los datos del certificado cuando la web utiliza el protocolo HTTPS.	7
2.5. Busca e indica un software con el que podríamos capturar los paquetes de la transmisión cuando se realiza la carga de una web. Así, por ejemplo, se podría ver cómo viaja la información no cifrada en el caso de HTTP y cómo viaja cifrada en el caso de HTTPS.	8

1. Tarea 1. Investiga y responde

1.1. ¿En qué arquitectura está basada el protocolo HTTP?

El protocolo HTTP (HyperText Transfer Protocol) está basado en una arquitectura cliente-servidor de tipo petición-respuesta (request-response). En este modelo, el cliente (normalmente un navegador web, cURL o una aplicación móvil) inicia la comunicación enviando un mensaje de petición HTTP al servidor. El servidor procesa dicha solicitud y devuelve un mensaje de respuesta HTTP con el contenido solicitado o una indicación del resultado del procesamiento.

HTTP es un protocolo sin estado (stateless), lo que significa que por defecto el servidor no conserva información de peticiones anteriores entre sesiones, recurriendo a mecanismos como cookies, tokens o sesiones para mantener el estado de los usuarios cuando es necesario.

1.2. ¿Qué tipos o categorías de códigos de estado existen? Enuméralos y explica brevemente en qué consisten.

Los códigos de estado HTTP son dígitos de tres cifras que se agrupan en 5 categorías según el primer dígito:

- **1XX (Información):** Indican que la petición ha sido recibida y el servidor continúa procesando el proceso. Son respuestas provisionales (ej. 100 Continue, 101 Switching Protocols).
- **2XX (Éxito):** Indican que la petición fue recibida, entendida y aceptada correctamente por el servidor (ej. 200 OK, 201 Created, 204 No Content).
- **3XX (Redirección):** Indican que el cliente debe realizar acciones adicionales para completar la solicitud, generalmente dirigirse a una nueva URL (ej. 301 Moved Permanently, 302 Found, 304 Not Modified).
- **4XX (Error del Cliente):** Indican que la solicitud contiene sintaxis incorrecta o no puede ser procesada debido a un problema achacable al cliente (ej. 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found).
- **5XX (Error del Servidor):** Indican que el servidor falló al intentar procesar una solicitud aparentemente válida (ej. 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable).

1.3. Indica algún ejemplo de web con error 4XX.

Un ejemplo clásico de error 4XX es el código 404 Not Found cuando se intenta acceder a una ruta inexistente dentro de un dominio activo. Por ejemplo:

- <https://www.google.com/holasoyjosemi>
- <https://github.com/holaholalol12312312312342>

Al navegar a estas direcciones, el servidor responde con un código de estado HTTP 404 Not Found, indicando que el cliente ha solicitado un recurso que no se encuentra alojado en el servidor.

1.4. Si obtenemos como respuesta HTTP "HTTP/1.1 302 Found", ¿qué tipo de error es y qué significa?

No es un error, sino una respuesta de redirección temporal perteneciente a la categoría 3XX (Redirección). Significa que el recurso solicitado ha sido movido temporalmente a una ubicación diferente especificada en la cabecera 'Location' de la respuesta HTTP. El cliente debe continuar realizando sus peticiones a la URL original en el futuro, pero redirigir la solicitud actual a la nueva dirección.

1.5. Si obtenemos como respuesta HTTP "HTTP/1.1 400 Bad Request", ¿qué tipo de error es y qué significa?

Es un error del cliente (categoría 4XX). Significa que el servidor no puede o no procesará la petición debido a que se percibe como un error del cliente (sintaxis de petición malformada, encuadre de mensaje de solicitud no válido, parámetros erróneos o enrutamiento de solicitud engañoso).

1.6. Si obtenemos como respuesta HTTP "HTTP/1.1 401 Authorization Required", ¿qué tipo de error es y qué significa?

Es un error del cliente (categoría 4XX). Significa que la solicitud carece de credenciales válidas de autenticación para el recurso solicitado. El servidor requiere que el usuario se autentique (mediante login con usuario/contraseña, HTTP Basic Auth o Bearer token) antes de conceder acceso.

2. Tarea 2. Inspección de las cabeceras HTTP

2.1. Qué cabeceras de respuesta del servidor se generan en cada caso y para qué sirven. Además, indica qué protocolo de transmisión utilizan una vez finalizada la carga del sitio web.

Web 1: TodoFP (Ministerio de Educación)

URL:

- <https://www.todofp.es/que-estudiar/familias-profesionales/informatica-Comunicaciones/des-aplicaciones-web.html>

Cabeceras destacadas:

- **Server:** Servidor web (Apache / Nginx / Akamai) que procesa la petición.
- **Content-Type:** Especifica el tipo MIME del contenido devuelto (text/html; charset=UTF-8).
- **Strict-Transport-Security (HSTS):** Fuerza el uso exclusivo de conexiones HTTPS seguras.
- **Cache-Control / Vary:** Define las políticas de almacenamiento en caché y compresión de respuesta.

Protocolo: HTTPS

Web 2: Portal Edu GVA

URL:

- <http://portal.edu.gva.es/03012785/es/inici-es/>

Cabeceras destacadas:

- **Location:** Si se accede por HTTP, genera un código 301/302 redirigiendo a [https://portal.edu.gva.es/...](https://portal.edu.gva.es/)
- **Server:** Servidor web institucional (Apache/IIS).
- **Set-Cookie:** Establece cookies de sesión o preferencia lingüística (e.g. PHPSESSID).
- **Content-Type:** text/html; charset=UTF-8.

Protocolo: HTTPS tras la redirección automática desde HTTP.

Web 3: Universidad Politécnica de Valencia (UPV)

URL:

- <http://www.upv.es/>

Cabeceras destacadas:

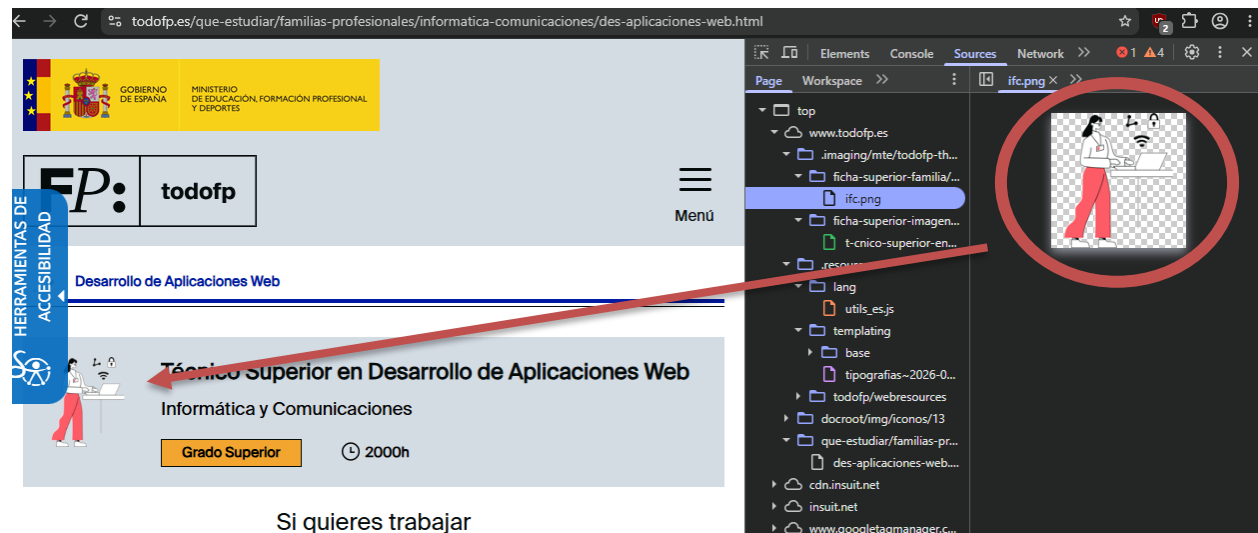
- **HTTP/1.1 301 Moved Permanently:** Redirige las conexiones sin cifrar a <https://www.upv.es/>.
- **Location:** <https://www.upv.es/>
- **Server:** Apache / Nginx.
- **X-Frame-Options / Content-Security-Policy:** Cabeceras de protección frente a clickjacking y XSS.
-

Protocolo: HTTPS de forma definitiva.

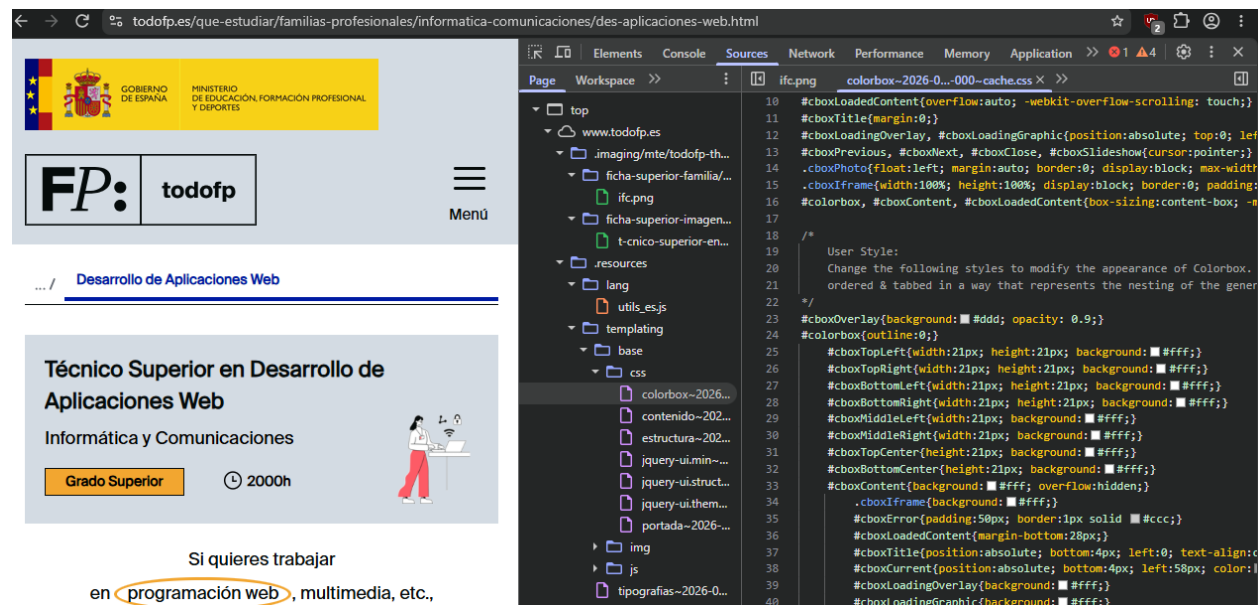
2.2. Las fuentes o "source" de las webs. Explica con tus palabras qué ves en la estructura de carpetas. Acompaña de alguna captura tu explicación.

Al inspeccionar la pestaña Sources (Fuentes) de las DevTools del navegador, se observa la estructura del árbol de archivos que conforman la aplicación web del cliente. En esta sección se agrupan los recursos descargados que se usan para mostrar la web al usuario.

Por ejemplo tenemos las carpetas donde se ubican las imágenes que salen en la web:



Otro ejemplo de lo que Podemos encontrar aqui es los archivos CSS que le dan estilo a la página:



2.3. Qué diferencia hay entre la información que viaja transmitida con el protocolo HTTP y el HTTPS.

La diferencia fundamental radica en la seguridad y el cifrado de la capa de transporte:

1. HTTP (HyperText Transfer Protocol): Transmite los datos en texto plano (sin cifrar). Cualquier intermediario en la red (mediante un ataque Man-in-the-Middle o sniffer) puede interceptar y leer credenciales, datos personales o formularios enviados.

2. HTTPS (HTTP Secure): Utiliza el protocolo SSL/TLS para establecer un canal cifrado entre el cliente y el servidor. La información viaja encriptada, garantizando tres pilares de seguridad:

- **Confidencialidad:** Nadie puede leer el contenido en tránsito.
- **Integridad:** Los datos no pueden ser alterados durante la transmisión.
- **Autenticidad:** Se verifica la identidad real del servidor mediante un certificado digital.

2.4. Indica en qué sección de la herramienta de inspección que utilices se pueden revisar los datos del certificado cuando la web utiliza el protocolo HTTPS.

En Google Chrome y Microsoft Edge, los datos del certificado SSL/TLS se pueden revisar de dos formas principales dentro de DevTools:

1. **Pestaña Security (Seguridad):** Muestra una visión general del estado de seguridad de la conexión y un botón explícito llamado 'View certificate' (Ver certificado) que despliega los detalles de la entidad emisora (CA), periodo de validez, algoritmo de cifrado y huella digital.
2. **Pestaña Network (Red):** Al seleccionar la petición principal y revisar la pestaña 'Overview' o el icono del candado en la barra de navegación del navegador.

2.5. Busca e indica un software con el que podríamos capturar los paquetes de la transmisión cuando se realiza la carga de una web. Así, por ejemplo, se podría ver cómo viaja la información no cifrada en el caso de HTTP y cómo viaja cifrada en el caso de HTTPS.

El software estándar de la industria para captura y análisis de paquetes de red es Wireshark.

- **En transmisiones HTTP (puerto 80):** Wireshark permite recomponer el flujo TCP (Follow TCP Stream) y ver las peticiones GET/POST, las cabeceras y el cuerpo HTML en texto plano legible.
- **En transmisiones HTTPS (puerto 443):** Wireshark solo muestra paquetes cifrados con el protocolo TLS/SSL (Handshake, Encrypted Application Data), resultando incomprensible el contenido de la comunicación a menos que se disponga de la clave privada o del registro de claves de sesión (TLS Key Log).

Otras alternativas destacadas incluyen tshark (versión en línea de comandos de Wireshark), Fiddler y Charles Proxy.